Summarize

Chapter 7. Sharded Services

In Chapter 6, we saw the value of replicating stateless services for reliability, redundancy, and scaling. This chapter considers sharded services. With the replicated services that we introduced in Chapter 6, each replica was entirely homogeneous and capable of serving every request. In contrast to replicated services, with sharded services, each replica, or *shard*, is only capable of serving a subset of all requests. A load-balancing node, or *root*, is responsible for examining each request and distributing each request to the appropriate shard or shards for processing. The contrast between replicated and sharded services is represented in Figure 7-1.

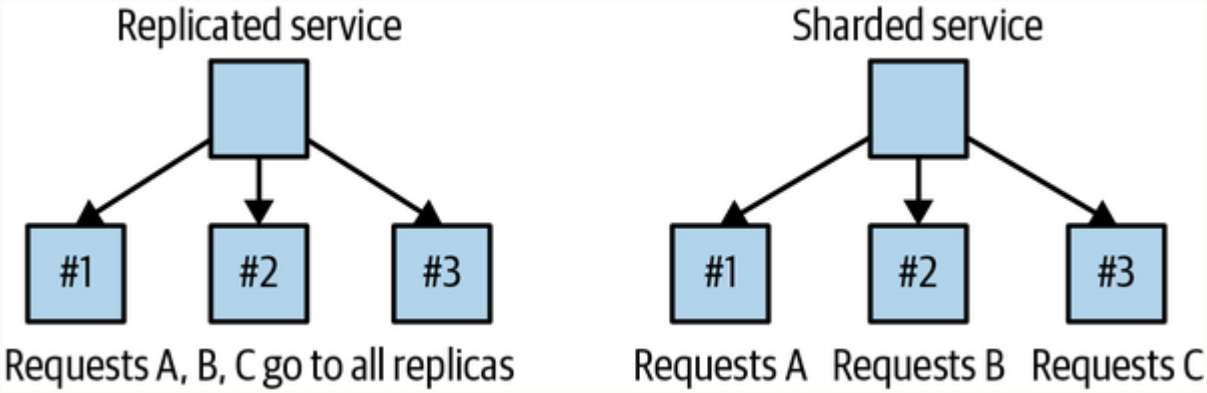


Figure 7-1. Replicated service versus sharded service

Replicated services are generally used for building stateless services, whereas sharded services are generally used for building stateful services. The primary reason for sharding the data is because the size of the state is too large to be served by a single machine. Sharding enables you to scale a service in response to the size of the state that needs to be served.

Sharding is not exclusively for stateful services, though. Sometimes sharding can also be useful for stateless services for the purpose of isolation. Failures in distributed systems can sometimes come from the input requests to the system ("poison requests") in a replicated system—if the poison requests come in a large enough volume, they can take down all of the replicas. In such systems, sharding can form an isolation boundary that protects the majority of the requests from the poison ones. Though the poison requests

may take out the entire shard, that will only be a fraction of the requests that the system processes.

Sharded Caching

To completely illustrate the design of a sharded system, this section provides a deep dive into the design of a sharded caching system. A *sharded cache* is a cache that sits between the user requests and the actual frontend implementation. A high-level diagram of the system is shown in Figure 7-2.

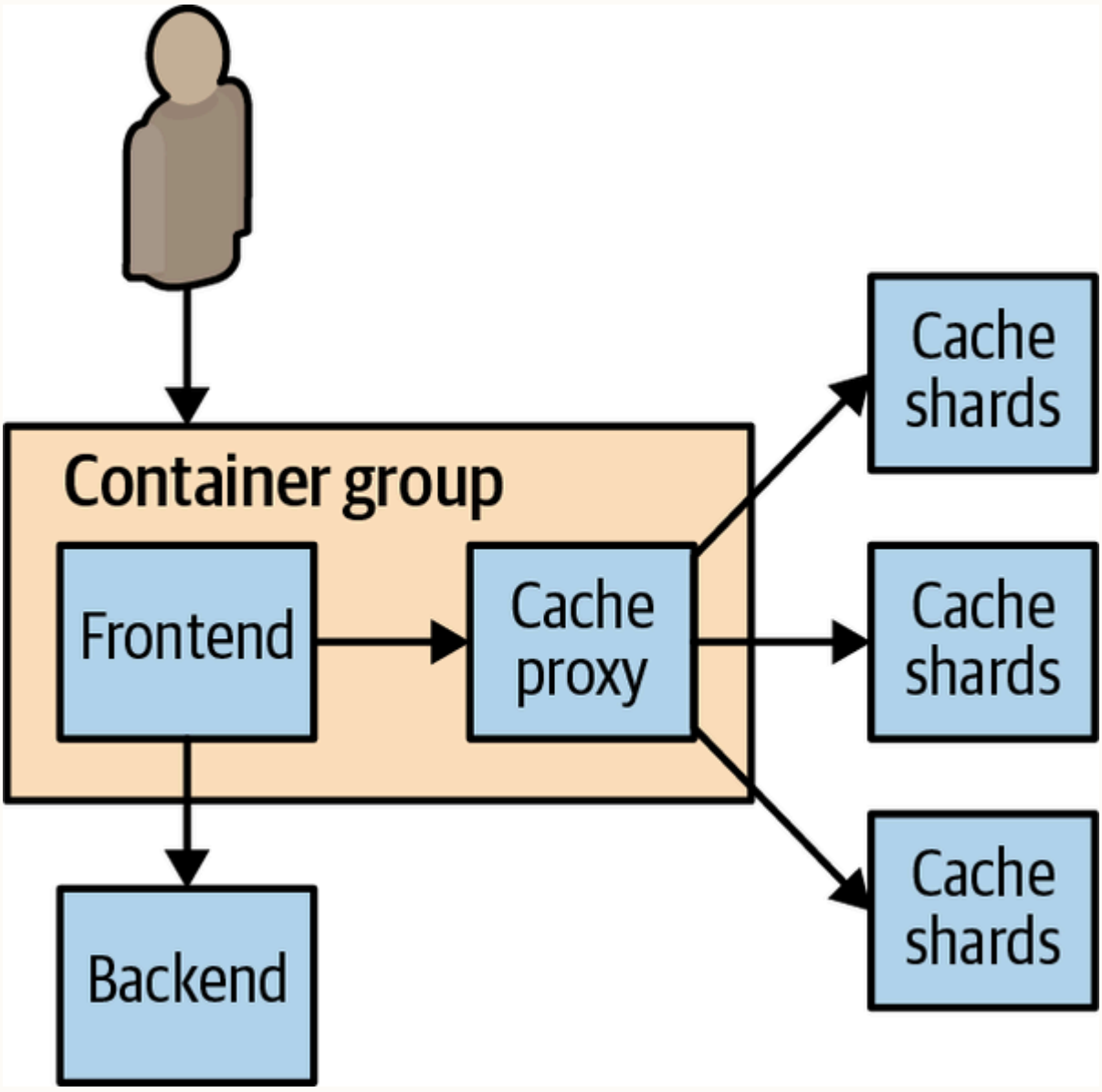


Figure 7-2. A sharded cache

In Chapter 4, we discussed how an ambassador could be used to distribute data to a sharded service. This section discusses how to build that service. When designing a sharded cache, there are a number of design aspects to consider:

- Why you might need a sharded cache
- The role of the cache in your architecture

- Replicated sharded caches
- The sharding function

Why You Might Need a Sharded Cache

Reminder: the primary reason for sharding any service is to increase the size of the data being stored in the service. To understand how this helps a caching system, imagine the following system: each cache has 10 GB of RAM available to store results, and can serve 100 requests per second (RPS). Suppose then that our service has a total of 200 GB possible results that could be returned, and an expected 1,000 RPS. Clearly, we need 10 replicas of the cache in order to satisfy 1,000 RPS (10 replicas × 100 requests per second per replica). The simplest way to deploy this service would be as a replicated service, as described in Chapter 6. But deployed this way, the distributed cache can only hold a maximum of 5% (10 GB/200 GB) of the total data set that we are serving. This is because each cache replica is independent, and thus each cache replica stores roughly the exact same data in the cache. This is great for redundancy, but pretty terrible for maximizing memory utilization. If instead, we deploy a 10-way sharded cache, we can still serve the appropriate number of RPS (10 × 100 is still 1,000), but because each cache serves a completely unique set of data, we are able to store 50% (10 × 10 GB/200 GB) of the total data set. This tenfold increase in cache storage means that the memory for the cache is much better utilized, since each key exists only in a single cache. The more data we store in the cache, the faster the overall system will be able to operate since more requests can be served from the cache.

The Role of the Cache in System Performance

In Chapter 6 we discussed how caches can be used to optimize end-user performance and latency, but one thing that wasn't covered was the criticality of the cache to your application's performance, reliability, and stability.

Put simply, the important question for you to consider is: if the cache were to fail, what would the impact be for your users and your service?

When we discussed the replicated cache, this question was less relevant because the cache itself was horizontally scalable, and failures of specific replicas would only lead to transient failures. Likewise, the cache could be horizontally scaled in response to increased load without impacting the end user.

This changes when you consider sharded caches. Because a specific user or request is always mapped to the same shard, if that shard fails, that user or request will always miss

the cache until the shard is restored. Given the nature of a cache as transient data, this miss is not inherently a problem, and your system must know how to recalculate the data. However, this recalculation is inherently slower than using the cache directly, and thus it has performance implications for your end users.

The performance of your cache is defined in terms of its *hit rate*. The hit rate is the percentage of the time that your cache contains the data for a user request. Ultimately, the hit rate determines the overall capacity of your distributed system and affects the overall capacity and performance of your system.

Imagine, if you will, that you have a request-serving layer that can handle 1,000 RPS. After 1,000 RPS, the system starts to return `HTTP 500` errors to users. If you place a cache with a 50% hit rate in front of this request-serving layer, adding this cache increases your maximum RPS from 1,000 RPS to 2,000 RPS. To understand why this is true, you can see that of the 2,000 inbound requests, 1,000 (50%) can be serviced by the cache, leaving 1,000 requests to be serviced by your serving layer. In this instance, the cache is fairly critical to your service, because if the cache fails, then the serving layer will be overloaded and half of all your user requests will fail. Given this, it likely makes sense to rate your service at a maximum of 1,500 RPS rather than the full 2,000 RPS. If you do this, then you can sustain a failure of half of your cache replicas and still keep your service stable.

But the performance of your system isn't just defined in terms of the number of requests that it can process. Your system's end-user performance is defined in terms of the *latency* of requests as well. A result from a cache is generally significantly faster than calculating that result from scratch. Consequently, a cache can improve the speed of requests as well as the total number of requests processed. To see why this is true, imagine that your system can serve a request from a user in 100 milliseconds. You add a cache with a 25% hit rate that can return a result in 10 milliseconds. Thus, the average latency for a request in your system is now 77.5 milliseconds. Unlike maximum requests per second, the cache simply makes your requests faster, so there is somewhat less need to worry about the fact that requests will slow down if the cache fails or is being upgraded. However, in some cases, the performance impact can cause too many user requests to pile up in request queues and ultimately time out. It's always recommended that you load test your system both with and without caches to understand the impact of the cache on the overall performance of your system.

Finally, it isn't just failures that you need to think about. If you need to upgrade or redeploy a sharded cache, you cannot just deploy a new replica and assume it will immediately take the existing load. A newly created cache shard doesn't have any data stored in

memory. Responses are cached as they are received in response to user requests. A newly deployed cache needs to be "warmed up" or filled with responses before it can provide maximal benefit. Deploying a new version of a sharded cache will generally result in temporarily losing some capacity. Another, more advanced option is to replicate your shards.

Replicated Sharded Caches

Sometimes your system is so dependent on a cache for latency or load that it is not acceptable to lose an entire cache shard if there is a failure or you are doing a rollout. Alternatively, you may have so much load on a particular cache shard that you need to scale it to handle the load. For these reasons, you may choose to deploy a sharded replicated service. A sharded replicated service combines the replicated service pattern described in Chapter 6 with the sharded pattern described in previous sections. In a nutshell, rather than having a single server implement each shard in the cache, a replicated service is used to implement each cache shard.

This design is obviously more complicated to implement and deploy, but it has several advantages over a simple sharded service. Most importantly, by replacing a single server with a replicated service, each cache shard is resilient to failures and is always present during failures. Rather than designing your system to be tolerant to performance degradation resulting from cache shard failures, you can rely on the performance improvements that the cache provides. Assuming that you are willing to over-provision shard capacity, this means that it is safe for you to do a cache rollout during peak traffic, rather than waiting for a quiet period for your service.

Additionally, because each replicated cache shard is an independent replicated service, you can scale each cache shard in response to its load; this sort of "hot sharding" is discussed at the end of this chapter.

Hands On: Deploying an Ambassador and Memcache for a Sharded Cache

In Chapter 4 we saw how to deploy a sharded Redis service. Deploying a sharded memcache is similar.

First, we will deploy memcache as a Kubernetes `StatefulSet`:

```
apiVersion: apps/v1
kind: StatefulSet
```

```
metadata:
  name: sharded-memcache
spec:
  serviceName: "memcache"
  replicas: 3
  template:
    metadata:
      labels:
        app: memcache
    spec:
      terminationGracePeriodSeconds: 10
      containers:
      - name: memcache
        image: memcached
        ports:
        - containerPort: 11211
          name: memcache
```

Save this to a file named *memcached-shards.yaml*, and you can deploy this with `kubectl create -f memcached-shards.yaml`. This will create three containers running memcached.

As with the sharded Redis example, we also need to create a Kubernetes `Service` that will create DNS names for the replicas we have created. The service looks like this:

```
apiVersion: v1
kind: Service
metadata:
  name: memcache
  labels:
    app: memcache
spec:
  ports:
  - port: 11211
    name: memcache
  clusterIP: None
  selector:
    app: memcache
```

Save this to a file named *memcached-service.yaml* and deploy it with `kubectl create -f memcached-service.yaml`. You should now have DNS entries for `memcache-0.memcache`, `memcache-1.memcache`, etc. As with Redis, we can use these names to configure `twemproxy`:

```
memcache:
  listen: 127.0.0.1:11211
  hash: fnv1a_64
  distribution: ketama
  auto_eject_hosts: true
  timeout: 400
  server_retry_timeout: 2000
  server_failure_limit: 1
  servers:
   - memcache-0.memcache:11211:1
   - memcache-1.memcache:11211:1
   - memcache-2.memcache:11211:1
```

In this config, you can see that we are serving the memcache protocol on `localhost:11211` so that the application container can access the ambassador. We will deploy this into our ambassador pod using a Kubernetes `ConfigMap` object that we can create with: `kubectl create configmap --from-file=nutcracker.yaml twem-config`.

Finally, all of the preparations are done, and we can deploy our ambassador example. We define a pod that looks like this:

```
apiVersion: v1
kind: Pod
metadata:
  name: sharded-memcache-ambassador
spec:
  containers:
    # This is where the application container would go, for example
    # - name: nginx
    #   image: nginx
    # This is the ambassador container
    - name: twemproxy
      image: ganomede/twemproxy
      command:
      - nutcracker
      - -c
      - /etc/config/nutcracker.yaml
      - -v
      - 7
      - -s
      - 6222
      volumeMounts:
      - name: config-volume
```

```
      mountPath: /etc/config
  volumes:
   - name: config-volume
     configMap:
       name: twem-config
```

You can save this to a file named *memcached-ambassador-pod.yaml*, and then deploy it with:

```
kubectl create -f memcached-ambassador-pod.yaml
```

Of course, we don't have to use the ambassador pattern if we don't want to. An alternative is to deploy a replicated *shard router* service. There are trade-offs between using an ambassador versus using a shard routing service. The value of the service is a reduction of complexity. You don't have to deploy the ambassador with every pod that wants to access the sharded memcache service; it can be accessed via a named and load-balanced service. The downside of a shared service is twofold. First, because it is a shared service, you will have to scale it larger as demand load increases. Second, using the shared service introduces an extra network hop that will add some latency to requests and contribute network bandwidth to the overall distributed system.

To deploy a shared routing service, you need to change the `twemproxy` configuration slightly so that it listens on all interfaces, not just localhost:

```
memcache:
  listen: 0.0.0.0:11211
  hash: fnv1a_64
  distribution: ketama
  auto_eject_hosts: true
  timeout: 400
  server_retry_timeout: 2000
  server_failure_limit: 1
  servers:
   - memcache-0.memcache:11211:1
   - memcache-1.memcache:11211:1
   - memcache-2.memcache:11211:1
```

You can save this to a file named *shared-nutcracker.yaml*, and then create a corresponding `ConfigMap` using `kubectl`:

```
kubectl create configmap --from-file=shared-nutcracker.yaml shared-twem-config
```

Then you can turn up the replicated shard routing service as a `Deployment`:

```
apiVersion: extensions/v1
kind: Deployment
metadata:
  name: shared-twemproxy
spec:
  replicas: 3
  template:
    metadata:
      labels:
        app: shared-twemproxy
    spec:
      containers:
      - name: twemproxy
        image: ganomede/twemproxy
        command:
        - nutcracker
        - -c
        - /etc/config/shared-nutcracker.yaml
        - -v
        - 7
        - -s
        - 6222
        volumeMounts:
        - name: config-volume
          mountPath: /etc/config
      volumes:
      - name: config-volume
        configMap:
          name: shared-twem-config
```

If you save this to *shared-twemproxy-deploy.yaml*, you can create the replicated shard router using `kubectl`:

```
kubectl create -f shared-twemproxy-deploy.yaml
```

To complete the shard router, we have to declare a load balancer to process requests:

```
kind: Service
apiVersion: v1
metadata:
  name: shard-router-service
spec:
  selector:
    app: shared-twemproxy
  ports:
    - protocol: TCP
      port: 11211
      targetPort: 11211
```

This load balancer can be created using `kubectl create -f shard-router-service.yaml`.

An Examination of Sharding Functions

So far we've discussed the design and deployment of both simple sharded and replicated sharded caches, but we haven't spent very much time considering how traffic is routed to different shards. Consider a sharded service where you have 10 independent shards. Given some specific user request *Req*, how do you determine which shard *S* in the range from zero to nine should be used for the request? This mapping is the responsibility of the *sharding function*. A sharding function is very similar to a hashing function, which you may have encountered when learning about hashtable data structures. Indeed, a bucket-based hashtable could be considered an example of a sharded service. Given both *Req* and *Shard*, then the role of the sharding function is to relate them together, specifically:

- *Shard = ShardingFunction(Req)*

Commonly, the sharding function is defined using a *hashing function* and the modulo (%) operator. Hashing functions are functions that transform an arbitrary object into an integer *hash*. The hash function has two important characteristics for our sharding:

Determinism

: The output should always be the same for a unique input.

Uniformity

: The distribution of outputs across the output space should be equal.

For our sharded service, determinism and uniformity are the most important characteristics. Determinism is important because it ensures that a particular request *R* always goes to the same shard in the service. Uniformity is important because it ensures that load is evenly spread between the different shards.

Fortunately for us, modern programming languages include a wide variety of high-quality hash functions. However, the outputs of these hash functions are often significantly larger than the number of shards in a sharded service. Consequently, we use the modulo operator (%) to reduce a hash function to the appropriate range. Returning to our sharded service with 10 shards, we can see that we can define our sharding function as:

- *Shard = hash(Req)* % 10

If the output of the hash function has the appropriate properties in terms of determinism and uniformity, those properties will be preserved by the modulo operator.

Selecting a Key

Given this sharding function, it might be tempting to simply use the hashing function that is built into the programming language, hash the entire object, and call it a day. The result of this, however, will not be a very good sharding function.

To understand this, consider a simple HTTP request that contains three things:

- The time of the request
- The source IP address from the client
- The HTTP request path (e.g., */some/page.html*)

If we use a simple object-based hashing function, `shard(request)`, then it is clear that `{12:00, 1.2.3.4, /some/file.html}` has a different shard value than `{12:01, 5.6.7.8, /some/file.html}`. The output of the sharding function is different because the client's IP address and the time of the request are different between the two requests. But of course, in most cases, the IP address of the client and the time of the request don't impact the response to the HTTP request. Consequently, instead of hashing the entire request object, a much better sharding function would be `shard(request.path)`. When we use `request.path` as the shard key, then we map both requests to the same shard, and thus the response to one request can be served out of the cache to service the other.

Of course, sometimes client IP *is* important to the response that is returned from the frontend. For example, client IP may be used to look up the geographic region that the

user is located in, and different content (e.g., different languages) may be returned to different IP addresses. In such cases, the previous sharding function `shard(request.path)` will actually result in errors, since a cache request from a French IP address may be served a result page from the cache in English. In such cases, the cache function is too *general*, as it groups together requests that do not have identical responses.

Given this problem, it would be tempting then to define our sharding function as `shard(request.ip, request.path)`, but this sharding function has problems as well. It will cause two different French IP addresses to map to different shards, thus resulting in inefficient sharding. This shard function is too *specific*, as it fails to group together requests that are identical. A better sharding function for this situation would be:

- `shard(country(request.ip), request.path)`

This first determines the country from the IP address, and then uses that country as part of the key for the sharding function. Thus, multiple requests from France will be routed to one shard, while requests from the United States will be routed to a different shard.

Determining the appropriate key for your sharding function is vital to the optimal design of a sharded system. Determining the correct shard key requires an understanding of the requests that you expect to see.

Consistent Hashing Functions

Setting up the initial shards for a new service is relatively straightforward: you set up the appropriate shards and the roots to perform the sharding, and you are off to the races. However, what happens when you need to change the number of shards in your sharded service? Such "re-sharding" is often a complicated process.

To understand why this is true, consider the sharded cache previously examined. Certainly, scaling the cache from 10 to 11 replicas is straightforward to do with a container orchestrator, but consider the effect of changing the scaling function from *hash(Req) % 10* to *hash(Req) % 11*. When you deploy this new scaling function, a large number of requests are going to be mapped to a different shard than the one they were previously mapped to. In a sharded cache, this is going to temporarily cause a dramatic increase in your *miss rate* until the cache is repopulated with responses for the new requests that have been mapped to that cache shard by the new sharding function. In the worst case, rolling out a new sharding function for your sharded cache will be equivalent to a complete cache failure.

To resolve these kinds of problems, many sharding functions use *consistent hashing functions*. Consistent hashing functions are special hash functions that are guaranteed to only remap *# keys / # shards*, when being resized to *# shards*. For example, if we use a consistent hashing function for our sharded cache, moving from 10 to 11 shards will only result in remapping < 10% (*K / 11*) keys. This is dramatically better than losing the entire sharded service.

Hands On: Building a Consistent HTTP Sharding Proxy

To shard HTTP requests, the first question to answer is what to use as the key for the sharding function. Though there are several options, a good general-purpose key is the request path as well as the fragment and query parameters (i.e., everything that makes the request unique). Note that this does *not* include cookies from the user or the language/location (e.g., EN_US). If your service provides extensive customization to users or their location, you will need to include them in the hash key as well.

We can use the versatile `nginx` HTTP server for our sharding proxy:

```
worker_processes  5;
error_log  error.log;
pid        nginx.pid;
worker_rlimit_nofile 8192;

events {
  worker_connections  1024;
}

http {
    # define a named 'backend' that we can use in the proxy directive
    # below.
    upstream backend {
        # Has the full URI of the request and use a consistent hash
        hash $request_uri consistent
        server web-shard-1.web;
        server web-shard-2.web;
        server web-shard-3.web;
    }

    server {
        listen localhost:80;
        location / {
            proxy_pass http://backend;
        }
```

```
    }
  }
```

Note that we chose to use the full request URI as the key for the hash and use the key word `consistent` to indicate that we want to use a consistent hashing function.

Sharded Replicated Serving

Most of the examples in this chapter so far have described sharding in terms of cache serving. But, of course, caches are not the only kinds of services that can benefit from sharding. Sharding is useful when considering any sort of service where there is more data than can fit on a single machine. In contrast to previous examples, the key and sharding function are not a part of the HTTP request, but rather some context for the user.

For example, consider implementing a large-scale multiplayer game. Such a game world is likely to be far too large to fit on a single machine. However, players who are distant from each other in this virtual world are unlikely to interact. Consequently, the world of the game can be *sharded* across many different machines. The sharding function is keyed off of the player's location so that all players in a particular location land on the same set of servers.

Hot Sharding Systems

Ideally the load on a sharded cache will be perfectly even, but in many cases this isn't true, and "hot shards" appear because organic load patterns drive more traffic to one particular shard.

As an example of this, consider a sharded cache for a user's photos; when a particular photo goes viral and suddenly receives a disproportionate amount of traffic, the cache shard containing that photo will become "hot." When this happens, with a replicated sharded cache, you can scale the cache shard to respond to the increased load. Indeed, if you set up autoscaling for each cache shard, you can dynamically grow and shrink each replicated shard as the organic traffic to your service shifts around. An illustration of this process is shown in Figure 7-3. Initially the sharded service receives equal traffic to all three shards. Then the traffic shifts so that Shard A is receiving four times as much traffic as Shard B and Shard C. The hot sharding system moves Shard B to the same machine as

Shard C, and replicates Shard A to a second machine. Traffic is now, once again, equally shared among replicas.

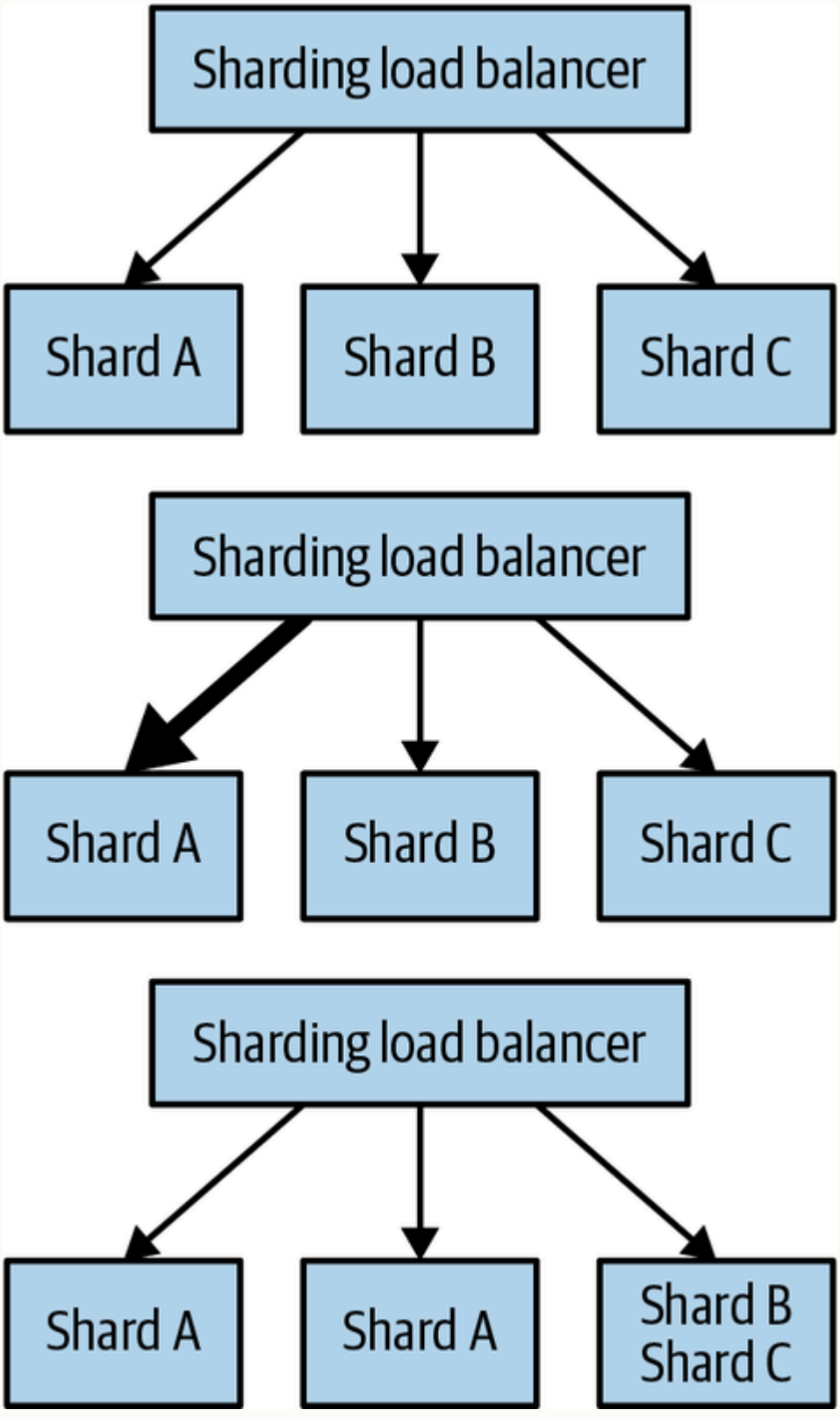


Figure 7-3. An example of a hot sharded system: initially the shards are evenly distributed, but when extra traffic comes to shard A, it is replicated to two machines, and shards B and C are combined on a single machine

Summary

This chapter introduced the concept of sharding, which directs particular requests to particular groups of machines instead of load-balancing all traffic to all replicas of a service. An example was given of how sharding can significantly improve the efficiency and

performance of a caching system attached to a serving API. Sharding is not just useful for caching, however. Systems like large-scale open world game servers and databases can scale beyond the size limits of a single machine by sharding their data across multiple different shard servers. Sharding also helps with isolation, limiting the blast radius of poison requests to a subset of all servers. Setting up a sharded system requires a careful understanding and design of both the shard key and the key hashing function.
Identifying the right way to partition your data can often make the difference between a scalable system and one which runs into bottlenecks as it scales. Understanding the details of sharding is critical to building larger and more complex distributed systems.

Previous chapter
< 6. Replicated Load-Balanced Services

Next chapter
8. Scatter/Gather >